

T.C.
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
BİL304 - İŞLETİM SİSTEMLERİ

DÖNEM PROJESİ SONUÇ RAPORU

*Contiki-NG Üzerinde OTA (Over-The-Air) Firmware Güncelleme Sisteminin
Simülasyonu*

Ders: BİL304 İşletim Sistemleri
Simülatör: Cooja (Contiki-NG)
Hedef Mimari: Texas Instruments MSP430 (Z1 Mote)
Tarih: Haziran 2025

1. GİRİŞ VE PROJENİN AMACI

Nesnelerin İnterneti (IoT) ekosisteminde, kısıtlı kaynaklara sahip gömülü sistemlerin uzaktan güncellenmesi kritik bir operasyonel gereksinim hâline gelmiştir. Sahaya konuşlandırılmış binlerce düğümü fiziksel olarak güncelleme yarattığı maliyet ve lojistik güçlükler, **OTA (Over-The-Air)** güncelleme yöntemlerini vazgeçilmez kılmaktadır. Bu proje, söz konusu gereksinimi akademik bir çerçevede ele alarak Contiki-NG işletim sistemi üzerinde çalışan Z1 mote'ları arasında kablosuz ağ üzerinden firmware (donanım yazılımı) güncelleme sürecini simüle etmektedir.

Projenin temel amacı; kaynak kısıtlı gömülü düğümler arasında güvenli, doğrulanabilir ve kesintisiz bir firmware transfer mekanizması tasarlamak ve bu mekanizmayı **Cooja simülatörü** ortamında gerçekçi biçimde test etmektir. Sistem, iki düğümlü bir mimariye dayanmaktadır:

- Node 2 (udp-client.c) — Gönderici/İstemci: Firmware verisini paketlere bölerek IPv6/RPL ağı üzerinden gönderen düğüm.
- Node 1 (udp-server.c) — Alıcı/Sunucu: Paketleri doğrulayan, Contiki Dosya Sistemi'ne (CFS) yazan ve güncellemeyi sonlandıran düğüm.

Bu rapor; kullanılan teknolojileri, sistem mimarisini, hata kontrol mekanizmalarını, derlenmiş ELF ikili dosyasının bellek analizini ve simülasyon sonuçlarını akademik bir çerçevede sunmaktadır.

2. KULLANILAN TEKNOLOJİLER VE KAVRAMLAR

2.1 Contiki-NG İşletim Sistemi

Contiki-NG, bellek ve işlem gücü açısından son derece kısıtlı gömülü sistemler için geliştirilmiş açık kaynaklı bir işletim sistemidir. Olay güdümlü (event-driven) çekirdek yapısı ve **Protothreads** programlama modeli sayesinde çoklu görev yönetimini minimal kaynak tüketimiyle gerçekleştirir. Düşük güç tüketimi, IPv6 desteği ve modüler ağ yığını (TSCH, 6LoWPAN, RPL) bu işletim sistemini IoT uygulamaları için endüstri standardı hâline getirmiştir.

2.2 Cooja Simülatörü

Cooja, Contiki-NG ekibi tarafından geliştirilen Java tabanlı bir ağ simülatörüdür. Gerçek donanımın yazılım modellemesini yaparak sanal mote'lar üzerinde Contiki-NG uygulamalarını çalıştırır. Bu proje kapsamında **Z1 mote donanım modeli** seçilmiştir; çünkü Z1, Texas Instruments MSP430F2617 mikrodenetleyicisi ve Chipcon CC2420 radyo alıcı-vericisini barındıran, gerçekçi kaynak kısıtlarını yansıtan bir platformdur.

2.3 IPv6/6LoWPAN ve RPL Yönlendirme Protokolü

6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks), standart IPv6 paketlerinin düşük bant genişlikli 802.15.4 ağlarında taşınabilmesi için paket sıkıştırma ve parçalama mekanizması sağlar. **RPL (Routing Protocol for Low-Power and Lossy Networks)**, bu ağlarda yönlendirmeyi yöneten IETF standardı bir protokoldür. RPL, düğümler arasında ağaç topolojisinde bir DODAG (Destination Oriented Directed Acyclic Graph) inşa ederek paketlerin hedefe ulaşmasını sağlar.

2.4 UDP İletişim Katmanı

Uygulama düzeyinde iletişim için UDP (User Datagram Protocol) tercih edilmiştir. TCP'nin görece ağır yapısı kaynak kısıtlı IoT düğümlerinde uygunsuz olduğundan, hata kontrolü ve yeniden gönderim mantığı uygulama katmanında (ACK/NACK mekanizması ile CRC doğrulaması aracılığıyla) geliştiriciler tarafından özel olarak implemente edilmiştir.

2.5 Contiki Dosya Sistemi (CFS)

CFS (Coffee File System), Contiki-NG bünyesindeki gömülü flash bellek tabanlı bir dosya sistemidir. Standart POSIX API'sine benzer bir arayüz sunan CFS, `cfs_open()`, `cfs_write()`, `cfs_read()` gibi fonksiyonlarla düğümlerin kalıcı depolama alanına erişmesini sağlar. Bu projede alıcı düğüm, doğrulanmış firmware bloklarını CFS üzerinde `"new-fw.z1"` isimli bir dosyaya sırasıyla yazmaktadır.

3. SİSTEM MİMARİSİ VE UYGULAMA

3.1 Genel Mimari

Sistem, iki ayrı Contiki-NG uygulaması (C kaynak dosyası) üzerine inşa edilmiştir. Her iki düğüm de ortak bir ağ katmanı (RPL + 6LoWPAN + UDP) üzerinden haberleşmekte olup iş mantığı uygulama katmanında tamamen birbirinden bağımsız yürütülmektedir.

Node 2 — Gönderici (udp-client.c)	Node 1 — Alıcı (udp-server.c)
- Firmware verisini 64-byte bloklara böler - Her blok için CRC-16 hesaplar - UDP ile Node 1'e gönderir - ACK/NACK bekler, NACK'te yeniden gönderir - Tüm bloklar bitince DONE paketi atar	- Gelen paketin CRC'sini doğrular - Doğruysa ACK, yanlışsa NACK döner - Bloğu CFS'e sırasıyla yazar - DONE alınca tüm imajı okur ve CRC doğrular - Başarıda "TEST OK" mesajı üretir

3.2 Gönderici Düğüm: udp-client.c

Gönderici düğümde firmware transfer döngüsü bir Protothread içinde çalışır. Her iterasyonda şu adımlar izlenir: Önce transfer edilecek blok belleğe alınır ve blok sırası (offset değeri) güncellenir. Ardından **blok başlığına gömlek sırası, toplam blok sayısı ve hesaplanan CRC-16 değeri** yazılır. Paket UDP soket arayüzü üzerinden Node 1'in IPv6 adresine gönderilir. Sistem, `MAX_RETRIES` sınırına ulaşana dek ACK bekleme döngüsünde kalır. ACK alındığında bir sonraki bloğa geçilir; NACK alındığında ise aynı blok yeniden gönderilir.

3.3 Alıcı Düğüm: udp-server.c

Alıcı düğüm, UDP paket dinleme olayına bağlı bir geri çağırım (callback) fonksiyonu ile çalışır. Paket geldiğinde sırasıyla şu işlemler gerçekleştirilir: Başlık alanları ayrıştırılır; beklenen blok sırası ile gelen offset değeri karşılaştırılır. CRC doğrulaması yapılır. Doğrulama başarılıysa veri `cfs_write()` ile dosyaya yazılır ve ACK paketi gönderilir. Başlık `"DONE"` dizesini içeriyorsa sistem tam imaj doğrulama moduna geçer.

4. HATA KONTROLÜ VE TAM İMAJ DOĞRULAMA

4.1 CRC-16 Algoritması

Projede hata denetimi için **CRC-16/IBM (Cyclic Redundancy Check, 16-bit)** algoritması kullanılmıştır. Bu algoritma, veri bütünlüğünü doğrulamak için IETF ve IEEE 802.15.4 standartlarında yaygın biçimde tercih edilmektedir. CRC hesaplama, `0xA001` polinomuyla bit düzeyinde XOR ve bit kaydırma işlemlerinin kombinasyonundan oluşur. Hesaplanan 16-bit değer, paketin başlığına eklenerek alıcıya iletilir.

Her 64-byte'lık bloğa ait CRC hesaplanır ve bu değer gönderici tarafından paket başlığına eklenir. Alıcı, gelen veri üzerinde bağımsız olarak aynı CRC hesaplamasını yürütür ve iki değeri karşılaştırır. Değerler eşleşmezse blok bozuk kabul edilir ve NACK gönderilir; eşleşirse blok kabul edilerek ACK iletilir.

4.2 ACK/NACK Mekanizması ve Yeniden Gönderim

Uygulama katmanında implemente edilen bu güvenilirlik mekanizması, UDP'nin doğası gereği sahip olmadığı paket doğrulama ve yeniden gönderim özelliğini sağlamaktadır. Gönderici, her bloğu ilettikten sonra `MAX_RETRIES` parametresiyle sınırlı bir bekleme döngüsüne girer. Süre dolmadan ACK alınırsa transfer devam eder. NACK alınırsa veya zaman aşımı gerçekleşirse aynı blok yeniden gönderilir. Bu mekanizma, geçici radyo karışıklıkları ve paket kayıplarına karşı sistemin dayanıklılığını artırmaktadır.

4.3 Tam İmaj Doğrulaması (Full Image Verification)

Tüm blokların başarıyla iletilmesinin ardından gönderici düğüm, alıcıya özel bir `"DONE"` paketi gönderir. Bu paketi alan alıcı düğüm, CFS'ye yazmış olduğu `"new-fw.z1"` dosyasını baştan sona okuyarak tüm firmware imajı üzerinden bir kez daha CRC-16 hesaplar. Hesaplanan değer, gönderici tarafından başlangıçta iletilmiş olan toplam CRC değeri ile karşılaştırılır.

Simülasyon testinde bu doğrulama işlemi beklenen sonucu üretmiş ve sistem **"Tüm imaj doğrulandı! CRC Eşleşiyor: 0x1C47"** çıktısını vererek güncellemeyi başarıyla tamamlamıştır. Bu iki kademeli doğrulama yaklaşımı (blok bazlı + imaj bazlı), olası veri bozulmalarına karşı güçlü bir bütünlük güvencesi sağlamaktadır.

5. DERLENMİŞ KOD (ELF) ANALİZİ VE BELLEK KULLANIMI

5.1 ELF Dosya Formatı ve Önemi

Derleme sonucunda üretilen `udp-server.z1` dosyası, ham ikili (raw binary) format yerine **ELF (Executable and Linkable Format)** yapısında üretilmiştir. ELF formatı; bölüm başlıkları, sembol tablosu, hata ayıklama bilgileri ve adres konumlandırma bilgilerini tek bir yapı içinde barındırır. Ham binary formatına kıyasla ELF, Cooja simülatörünün çalışma zamanında sembolik hata ayıklama yapmasına, bellek haritasının analiz edilmesine ve ağ paket izlemenin kaynak kod düzeyinde yorumlanmasına olanak tanır.

5.2 ELF Başlık Bilgileri

Derlenmiş dosyaya ait başlık bilgileri aşağıdaki tabloda özetlenmektedir:

ELF Sınıfı	ELF32 (32-bit mimariye uyumlu)
Hedef Mimari	Texas Instruments MSP430
Giriş Adresi (Entry Point)	0x3100 — Programın çalışmaya başladığı adres
Byte Sırası	Little-Endian
ELF Versiyonu	1 (Geçerli)

5.3 Bellek Bölümleri (Sections) Analizi

MSP430 mimarisi, **Flash (ROM)** ve **SRAM (RAM)** olmak üzere iki ayrı bellek bölgesine sahiptir. ELF bölümleri bu fiziksel bellek alanlarına doğrudan karşılık gelmektedir.

Bölüm Adı	Adres	Boyut	İşlev
<code>.text</code>	0x3100	~47 KB	Yürütülebilir makine kodları. ROM/Flash bellekte konumlanır; başlangıç adresi giriş noktasıyla örtüşür.
<code>.rodata</code>	0xe8a8	~1.7 KB	Salt okunur sabitler (string literalleri, sabit diziler). Flash'ta durur, RAM'e taşınmaz.
<code>.data</code>	0x1100	350 Byte	Başlangıç değeri atanmış global/statik değişkenler. Flash'ta saklanır, başlangıçta RAM'e (0x1100) kopyalanır.
<code>.bss</code>	0x125e	~6 KB	Başlangıç değeri atanmamış değişkenler. Fiziksel Flash alanı tüketmez; yalnızca RAM'de yer ayırır.
<code>.vectors</code>	0xffc0	64 Byte	MSP430 kesme (interrupt) vektör tablosu. Her kesme türüne ait işleyici (handler) fonksiyonunun adresi bu alanda tutulur.

5.4 Sembol Tablosu (.symtab) ve Hata Ayıklama

Sembol tablosu `.symtab`, derleme süreci boyunca üretilen tüm fonksiyon ve değişken isimlerini bunların bellek adresleriyle ilişkilendirir. Bu tablo sayesinde Cooja simülatörü; `cfs_write()`, `udp_new()`, `crc16_add()` gibi fonksiyonların hangi bellek adresinde çalıştığını takip edebilmekte, çalışma zamanı hataları kaynak kod satırlarına bağlanabilmekte ve simülasyon logları sembolik isimlerle yorumlanabilmektedir.

Bu analizin genel sonucu şöyledir: Sistem toplamda **~47 KB Flash** ve yaklaşık **~6.35 KB RAM** (`.data` + `.bss`) kullanmaktadır. Z1 mote'unun 92 KB Flash ve 8 KB RAM kapasitesi göz önüne alındığında, bu değerler hedef platformun kısıtları içinde makul bir kullanım verimliliğine işaret etmektedir.

6. SİMÜLASYON SONUÇLARI VE DEĞERLENDİRME

6.1 Test Ortamı

Sistem, Cooja simülatöründe iki Z1 mote tanımlanarak test edilmiştir. Node 1 (sunucu) ve Node 2 (istemci), sanal IEEE 802.15.4 radyo kanalı üzerinden haberleşmiş; RPL protokolü otomatik olarak ağı yapılandırmış ve yönlendirme tablosunu oluşturmuştur.

6.2 Simülasyon Çıktıları

10 dakikalık simülasyon süresi boyunca kaydedilen Cooja log çıktıları sistemin tüm bileşenlerinin beklenen şekilde çalıştığını doğrulamıştır. Gözlemlenen kritik log olayları aşağıda listelenmiştir:

- RPL ağının kurulması ve Node 2'nin Node 1'e ulaşabilir hâle gelmesi.

- Her 64-byte'lık bloğun gönderilmesi ve ACK alındığının loglanması.
- Simüle edilmiş bozuk paket senaryosunda NACK üretilmesi ve bloğun başarıyla yeniden gönderilmesi.
- DONE paketi alındıktan sonra tam imaj CRC hesaplamasının başlatılması.
-
-

6.3 Değerlendirme

Simülasyon sonuçları, tasarlanan OTA güncelleme mimarisinin **%100 başarı oranıyla** çalıştığını ortaya koymaktadır. İki kademeli CRC doğrulama mekanizması (blok düzeyi + imaj düzeyi), olası radyo kayıplarına ve veri bozulmalarına karşı sistemin güvenilirliğini etkin biçimde sağlamıştır. ACK/NACK ile yeniden gönderim mekanizması, UDP'nin sunmadığı güvenilirlik katmanını başarıyla uygulama düzeyinde karşılamıştır. CFS entegrasyonu sayesinde güncelleme süreci bir kesinti yaşansa bile devam edebilecek yapısal bir dayanıklılık kazanmıştır.

7. SONUÇ

Bu proje, kaynak kısıtlı IoT düğümleri için kapsamlı bir OTA firmware güncelleme sisteminin Contiki-NG ve Cooja ortamında başarıyla tasarlanıp test edildiğini göstermektedir. Sistem; **firmware parçalama, CRC-16 hata denetimi, ACK/NACK yeniden gönderim ve CFS tabanlı kalıcı depolama** bileşenlerini bir bütün olarak sunmaktadır.

ELF ikili analizi, projenin hedef MSP430 mimarisine uygun derlendiğini ve bellek kullanımının Z1 platformunun kısıtları içinde verimli kaldığını teyit etmektedir. .text, .bss, .data ve .vectors bölümlerinin ayrıştırılması, gömülü sistemlerde bellek yönetiminin somut bir örneğini sunmaktadır.

Cooja simülasyonu, 10 dakikalık çalışma süresi boyunca **TEST OK** çıktısıyla tüm fonksiyonel gereksinimlerin eksiksiz karşılandığını kanıtlamıştır. Bu sonuçlar, benzer OTA altyapılarının gerçek donanım platformlarına aktarılmasına yönelik sağlam bir akademik ve teknik temel oluşturmaktadır.